4.6 Eliminating Tail Recursion

The main benefit of a recursive approach to algorithm design is that it allows us to succinctly take advantage of a repetitive structure present in many problems. By making our algorithm description exploit the repetitive structure in a recursive way, we can often avoid complex case analyses and nested loops. This approach can lead to more readable algorithm descriptions, while still being quite efficient.

However, the usefulness of recursion comes at a modest cost. In particular, the Python interpreter must maintain activation records that keep track of the state of each nested call. When computer memory is at a premium, it is useful in some cases to be able to derive nonrecursive algorithms from recursive ones.

In general, we can use the stack data structure, which we will introduce in Section 6.1, to convert a recursive algorithm into a nonrecursive algorithm by managing the nesting of the recursive structure ourselves, rather than relying on the interpreter to do so. Although this only shifts the memory usage from the interpreter to our stack, we may be able to reduce the memory usage by storing only the minimal information necessary.

Even better, some forms of recursion can be eliminated without any use of axillary memory. A notable such form is known as ***tail recursion***. A recursion is a tail recursion if any recursive call that is made from one context is the very last operation in that context, with the return value of the recursive call (if any) immediately returned by the enclosing recursion. By necessity, a tail recursion must be a linear recursion (since there is no way to make a second recursive call if you must immediately return the result of the first).

Of the recursive functions demonstrated in this chapter, the binary_search function of Code Fragment 4.3 and the reverse function of Code Fragment 4.10 are examples of tail recursion. Several others of our linear recursions are almost like tail recursion, but not technically so. For example, our factorial function of Code Fragment 4.1 is *not* a tail recursion. It concludes with the command:

```
return n * factorial(n-1)
```

This is not a tail recursion because an additional multiplication is performed after the recursive call is completed. For similar reasons, the linear_sum function of Code Fragment 4.9 and the good_fibonacci function of Code Fragment 4.7 fail to be tail recursions.

Any tail recursion can be reimplemented nonrecursively by enclosing the body in a loop for repetition, and replacing a recursive call with new parameters by a reassignment of the existing parameters to those values. As a tangible example, our binary_search function can be reimplemented as shown in Code Fragment 4.15. We initialize variables low and high, just prior to our while loop, to represent the full extent of the sequence. Then, during each pass of the loop, we either find

```
def binary_search_iterative(data, target):
  """Return True if target is found in the given Python list."""
  low = 0
  high = len(data)-1
  while low <= high:
    mid = (low + high) // 2
    if target == data[mid]:                # found a match
      return True
    elif target < data[mid]:
      high = mid - 1                       # only consider values left of mid
    else:
      low = mid + 1                        # only consider values right of mid
  return False                             # loop ended without success
```

Code Fragment 4.15: A nonrecursive implementation of binary search.

the target, or we narrow the range of the candidate subsequence. Where we made the recursive call binary_search(data, target, low, mid -1) in the original version, we simply replace high = mid $-$ 1 in our new version and then continue to the next iteration of the loop. Our original base case condition of low $>$ high has simply been replaced by the opposite loop condition **while** low $<=$ high. In our new implementation, we return False to designate a failed search if the while loop ends (that is, without having ever returned True from within).

We can similarly develop a nonrecursive implementation (Code Fragment 4.16) of the original recursive reverse method of Code Fragment 4.10.

```
def reverse_iterative(S):
  """Reverse elements in sequence S."""
  start, stop = 0, len(S)
  while start < stop - 1:
    S[start], S[stop-1] = S[stop-1], S[start]      # swap first and last
    start, stop = start + 1, stop - 1              # narrow the range
```

Code Fragment 4.16: Reversing the elements of a sequence using iteration.

In this new version, we update the values start and stop during each pass of the loop, exiting once we reach the case of having one or less elements in that range.

Many other linear recursions can be expressed quite efficiently with iteration, even if they were not formally tail recursions. For example, there are trivial nonrecursive implementations for computing factorials, summing elements of a sequence, or computing Fibonacci numbers efficiently. In fact, our implementation of a Fibonacci generator, from Section 1.8, produces each subsequent value in $O(1)$ time, and thus takes $O(n)$ time to generate the nth entry in the series.